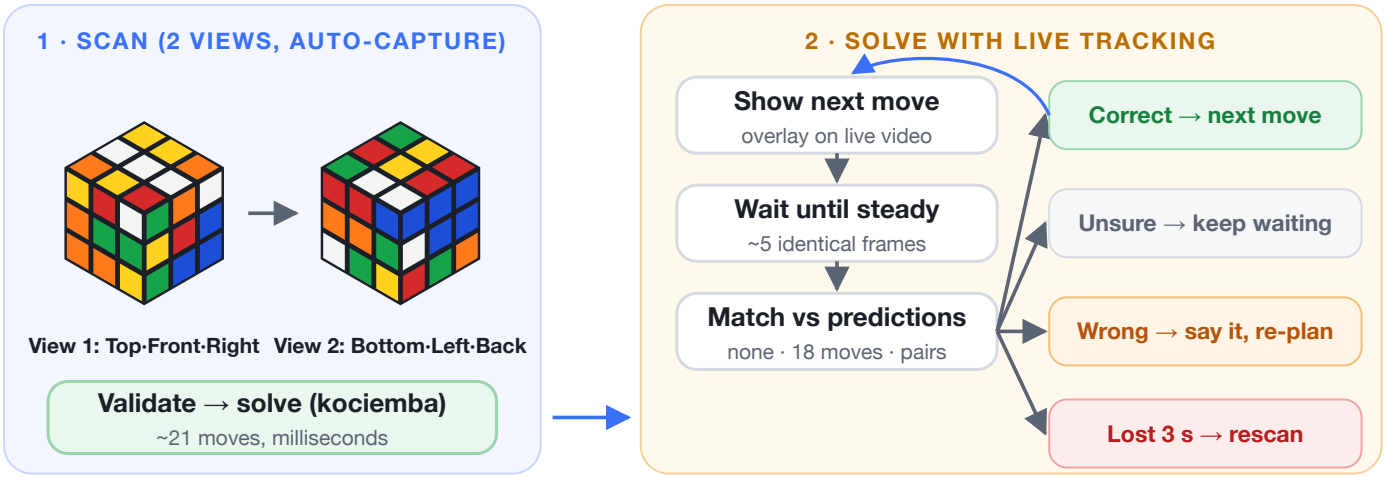
RUBIK'S CUBE SOLVER · ENGINEERING

Live Cube Solver: Implementation Plan

A webcam watches the user's cube. After one quick scan the app computes the solution, then follows every move live. A correct move advances the guide; a wrong move is identified, the solution is re-planned from the real cube state, and the user is told what to do next. A rescan is requested only if tracking is lost.

This plan is Phase 1: everything runs on a laptop with its webcam. Once it works, Phase 2 moves the same pipeline to our XREAL One Pro glasses with the XREAL Eye camera add-on.



Why these choices: measured, not guessed

SOLVER
20.7 moves
kociemba, 11 ms median · today: 110 moves

COLOUR NAMING
98% cubes right
normal light · 88.7% in hard conditions

FINDING THE CUBE
100% found
YOLO pose · 5.2% point error · 15 ms GPU / 40 ms CPU

LIVE TRACKING
98.7% correct
0 wrong moves named in 600 trials · 90.3% in hard conditions

From our benchmark of the open-source candidates (synthetic test frames + one real photo set; see page 8 and Benchmark_Report.pdf).

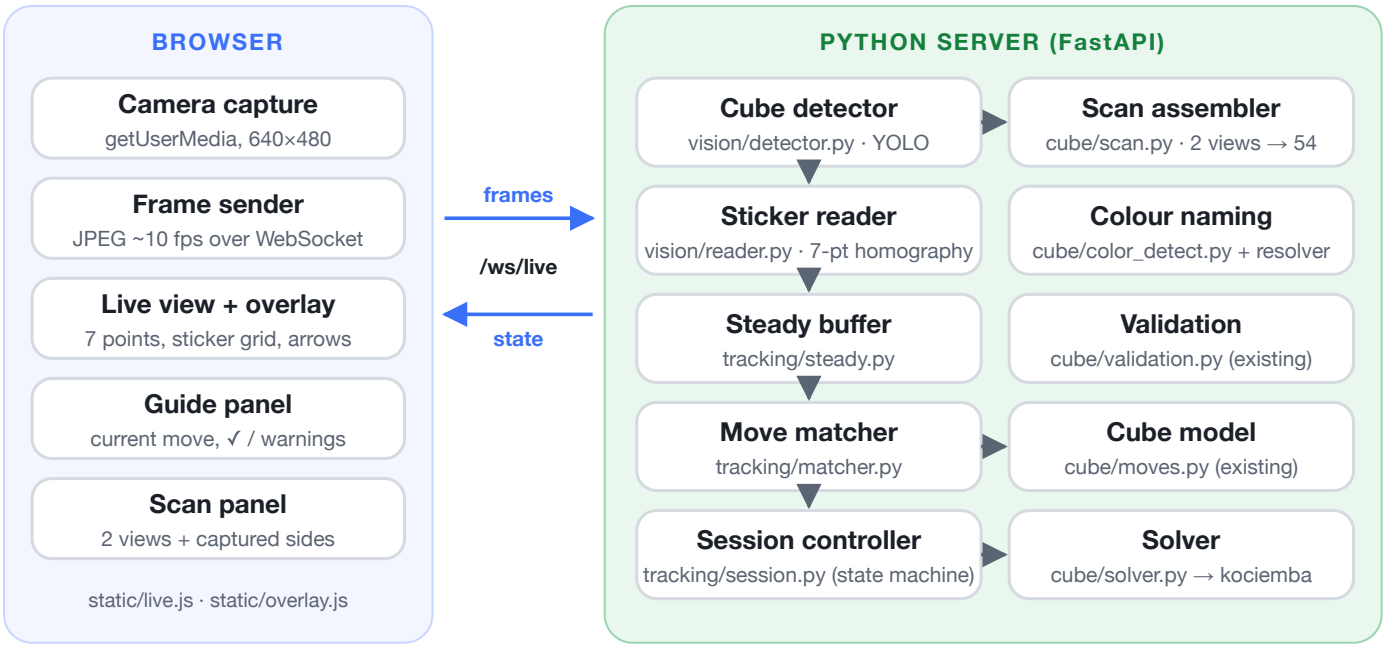
What changes compared with today's app

	TODAY	NEW
Scanning	6 faces, Capture button, line up with a grid	2 corner views, captured automatically when steady
Solver	Layer-by-layer, ~110 moves	kociemba two-phase, ~21 moves
Colours	Centre-based rebalance	Same + rubiks-color-resolver fallback when invalid
During the solve	User taps "next"; mistakes go unnoticed	Camera checks every move; wrong moves trigger a re-plan
Cube detection	None (fixed grid)	YOLO pose model: 7 corner points per frame

ARCHITECTURE

System overview

The browser only captures video and draws. All vision, tracking and solving runs in the Python server, so it can be reused in Phase 2 on our XREAL glasses with the Eye camera.

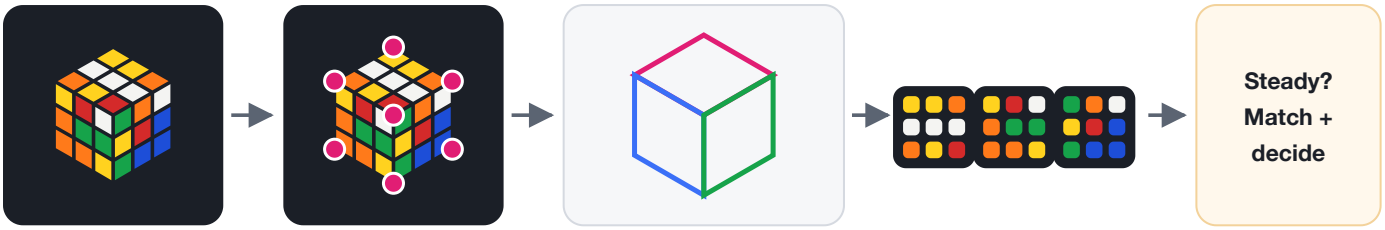


MODULE	LOCATION	RESPONSIBILITY	STATUS
WebSocket endpoint	main.py · /ws/live	Receive frames, run the pipeline, send state back	New
Cube detector	vision/detector.py	YOLO pose → 7 corner points + confidence	New
Sticker reader	vision/reader.py	3 homographies → 27 sticker colours (Lab, median of cell centre)	New
Steady buffer	tracking/steady.py	Accept a reading only after N consistent frames	New
Move matcher	tracking/matcher.py	Score readings against every predicted outcome	New
Session controller	tracking/session.py	Phase/state machine: scan → solve → track → rescan	New
Scan assembler	cube/scan.py	Map 2 corner views to the 54 facelets	New
Colour naming	cube/color_detect.py	Add rubiks-color-resolver fallback	Change
Solver	cube/solver.py	Wrap kociemba; keep the enrich() move descriptions	Change
Cube model + validation	cube/moves.py, cube/validation.py	Unchanged; reused by matcher and scan	Existing
Live page	static/live.js, overlay.js	Camera, frame sending, overlay, guide UI	New
Label tool	static/label.html	Drag 7 points to correct YOLO pre-labels	New
Benchmark + renderer	bench/	Synthetic frames, regression benchmarks, training data	Move in

PER-FRAME PIPELINE

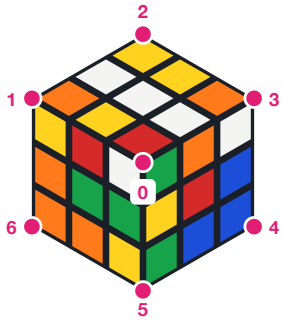
What happens to every camera frame

Each frame is a stateless function from pixels to 27 sticker colours. Only the tracker keeps state.



1 · Frame | 2 · YOLO: 7 points | 3 · 3 homographies | 4 · 27 sticker colours | 5 · Tracker

Keypoint order (the model's output contract)



Face quads from the 7 points

FACE	CORNERS (TL, TR, BR, BL)
Top (U)	2, 3, 0, 1
Front (F)	1, 0, 5, 6
Right (R)	0, 3, 4, 5

0 = corner nearest the camera (Front-Right-Top), then 1 FLU, 2 BLU, 3 BRU, 4 BRD, 5 FRD, 6 FLD. Each sticker is sampled as the median colour of the middle 40% of its cell (ignores glare specks and black borders), converted to Lab.

Latency budget

STAGE	MEASURED	TARGET	NOTES
YOLO pose (yolo26n-pose, 640 px)	15 ms GPU (M1 Pro) · 40 ms CPU	≤ 40 ms	Runs every frame; can skip frames under load
Sticker reader + move matcher	~62 ms (prototype)	≤ 10 ms	Precompute move permutations as numpy index arrays instead of applying moves in Python
kociemba re-plan	11 ms median	—	Only after a confirmed wrong move
End-to-end per frame	~130 ms (prototype)	≤ 100 ms (≥ 10 fps)	Browser sends the next frame only after a reply (no queue build-up)

WebSocket protocol

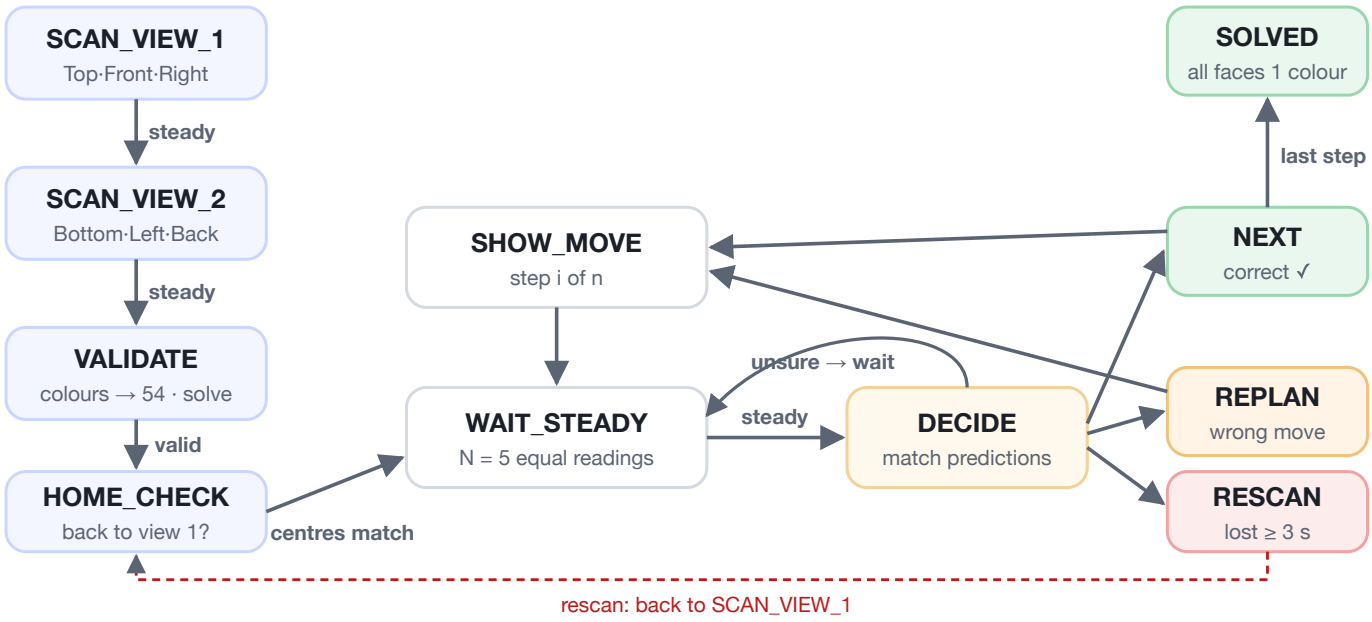
Browser → server (binary message): 4-byte frame id + JPEG bytes (640×480, quality ~80)
Server → browser (JSON):

```
{ "frame": 812, "phase": "tracking",              // scan_view_1 | scan_view_2 | solving | tracking | rescan | solved
  "keypoints": [[x,y], … 7 points] | null,  "confidence": 0.97,
  "stickers": {"U": ["white", …9], "F": […], "R": […]},
  "status": "waiting_steady",                 // waiting_steady | correct | wrong_move | unsure | lost
  "step": {"index": 7, "total": 21, "move": "R'", "description": "Turn the right side down"},
  "message": "You did L instead of R. New plan: 19 moves." }
```

SESSION LOGIC

Tracking state machine

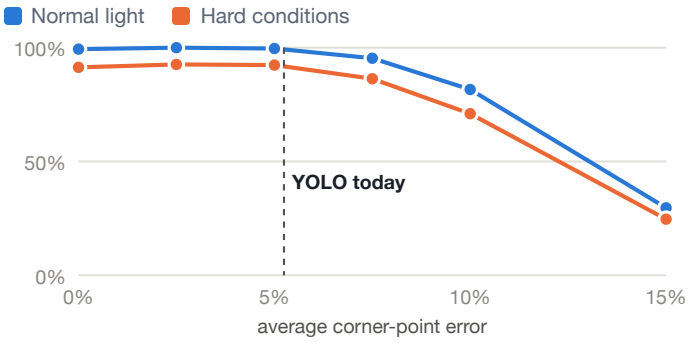
One controller owns the phase. Every frame either keeps the state or moves it along one arrow.



Decision rules

RULE	VALUE (FROM BENCHMARK)	WHY
Steady reading	5 consecutive frames with the same 27 colours and keypoints moving < 3% of cube size	Ignores half-done turns and fingers passing over
Candidates	No move, all 18 single moves, all 2-move pairs on different faces (≈ 340 states)	Covers a quick double move; shown to be unambiguous from Top+Front+Right
Score	Sum over 27 stickers of colour distance (ΔE, capped at 35) to the predicted colour	Cap stops one glared or covered sticker dominating
Priors	expected move 0 · other single move +4 · pair +12	Prefer the simplest explanation
Reference colours	Re-calibrated from the last confirmed steady frame	Follows lighting changes during the solve
Unsure	Best and second-best differ by < 6	Wait for more frames rather than guess
Lost	Best score > 22 per sticker, or no keypoints, for ≥ 3 s	Only then ask for a rescan

How accurate must the 7 points be?



Reading the chart

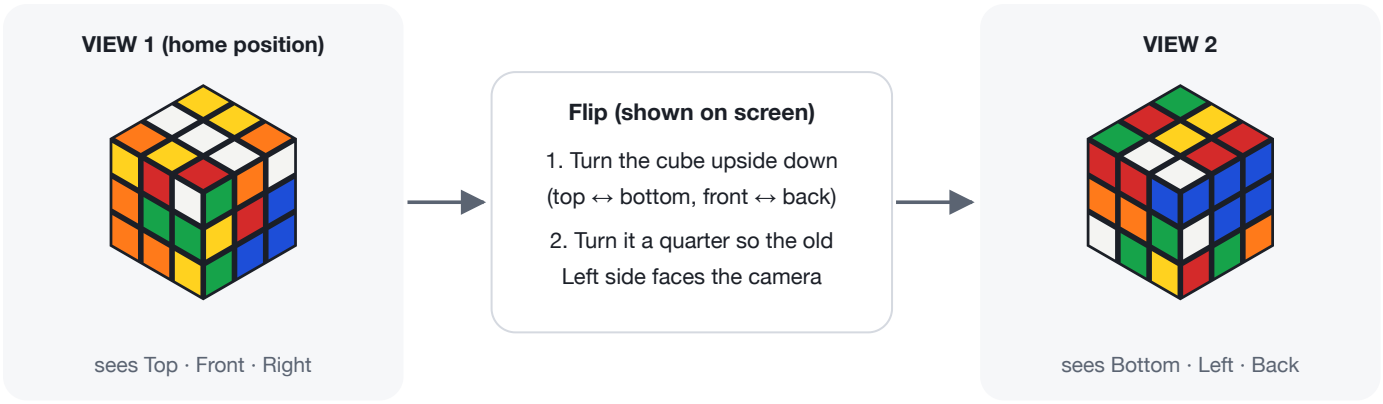
Tracking stays ~99% correct (normal light) while the average point error is ≤ 5% of the cube size, then drops fast. Wrong moves are almost never named, even when error is high: the tracker says "unsure" or "lost" instead.

Target for the YOLO model on real frames: average error < 5%. Today's synthetic-trained model sits right at that line.

SCAN

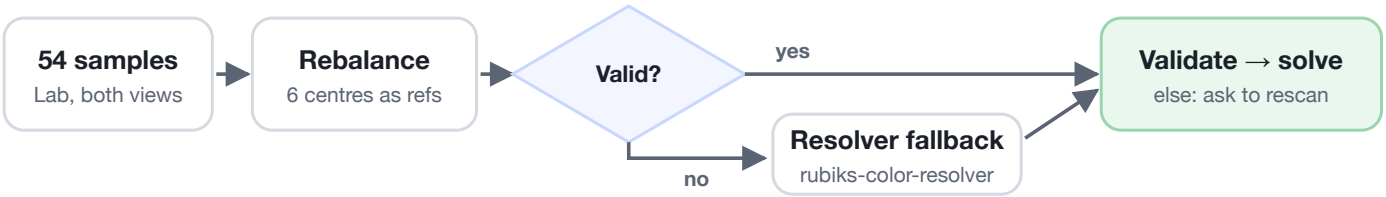
Scanning in 2 views

Each corner view shows 3 sides, so 2 views cover the cube. The same detector and reader are used for scanning and tracking.



The facelet mapping for view 2 (which image row/column becomes which of the 54 facelets) is derived from the cube model and must be verified by a renderer test before the UI is built (Step 4).

Colour naming



Benchmark (150 whole cubes per lighting level): this combination got **98%** of cubes fully right in normal light and **88.7%** in hard conditions, better than either method alone (96% / 94% in normal light).

Scan checks

CHECK	WHEN	USER SEES
Same view twice	View 2 centres equal view 1 centres	"That is the same side again: flip the cube as shown"
Opposite centres	The 3 view-2 centres must be the opposites of the 3 view-1 centres (white↔yellow, red↔orange, green↔blue)	"Flip doesn't look right: try again"
Colour counts / legal cube	After rebalance and fallback (existing validation)	Per-colour counts, then rescan the view that looks off
Back in home position	Before tracking starts: visible centres must equal view 1	"Turn the cube back to the first position"

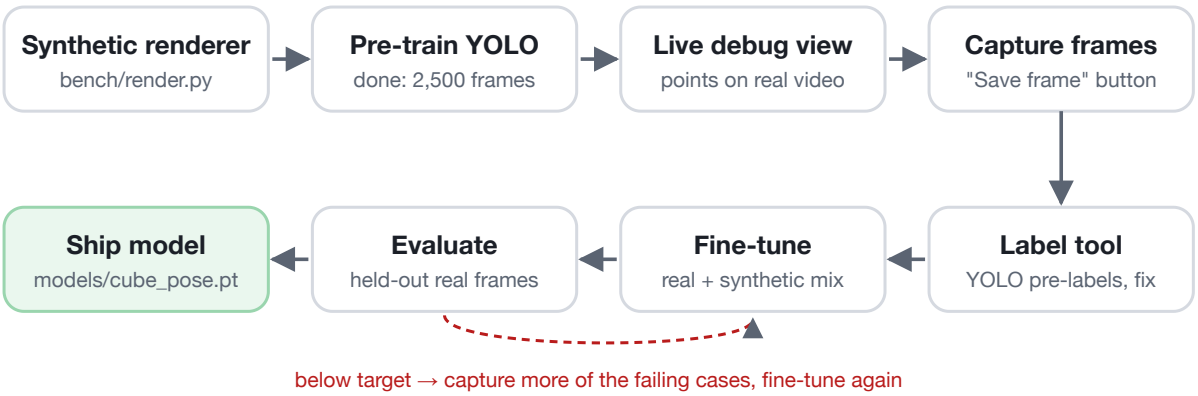
Auto-capture rule

A view is captured when YOLO confidence > 0.8 and the same 27 colours are read for ~0.5 s (5 frames). The captured view appears in the bottom panel; the user can tap it to redo it.

MODEL

YOLO data and training plan

The detector is the biggest risk: it has only been trained and tested on synthetic frames. Real frames of our cubes come first.



Dataset and targets

ITEM	SPECIFICATION
Model	Ultralytics YOLO pose, yolo26n-pose (nano), 640 px input
Keypoints	7, in the fixed order on page 3; kpt_shape [7, 3]; flip_idx [0, 3, 2, 1, 6, 5, 4]
Labels	One "cube" box + 7 points per image (Ultralytics pose txt format)
Real data	300–500 frames of our cubes: different angles, distances, lighting (daylight, lamp, dim), hands in all grips, a few motion-blurred frames
Split	80% train / 20% held-out test, split by recording session (not by frame) so the test is honest
Mix	Real frames + the synthetic set (keeps rare poses covered)
Targets on real test set	Cube found ≥ 98% · average point error < 5% of cube size · ≤ 40 ms on CPU
Export (later)	ONNX for the XREAL / Unity port

Current model (synthetic only)

	NORMAL LIGHT	HARD CONDITIONS
Cube found	100%	100%
Median point error	5.2%	5.1%
Tracking correct / wrong	98.7% / 0.0%	90.3% / 0.0%

Trained 8 passes (~32 min on an M1 Pro) on 2,500 synthetic frames; tested on 300 unseen synthetic frames per lighting level. Real-world performance is unknown until Step 3.

ROADMAP

Build order

Each step leaves the app working. Step 1 is shared groundwork; after it, the three workstreams run in parallel.

1 — Foundations
Move bench/ into the repo; kociemba solver; colour resolver fallback
Done when: 200-cube solver test ≤ 22 moves; all existing tests pass
Backend · Small

2 — Live loop + debug view
/ws/live endpoint; YOLO on every frame; overlay of 7 points + 27 colours
Done when: Points follow the real cube at ≥ 10 fps
All · Medium

3 — Real data + fine-tune
Save-frame button; label tool; fine-tune; real test set
Done when: Real test set: found ≥ 98%, error < 5%
Vision/ML · Large

4 — Two-view scan
scan.py facelet mapping; auto-capture; scan checks; UI
Done when: ≥ 95% of synthetic 2-view scans fully right; works on our cubes
Backend + Frontend · Medium

5 — Live tracking
steady buffer, matcher (vectorised), session state machine, re-plan, rescan; guide UI
Done when: Synthetic solve video with mistakes: 0 wrong moves named; real solves complete
Backend + Frontend · Large

6 — Polish, then Phase 2
Turn arrows drawn on the cube faces; sounds. Then Phase 2: XREAL glasses with the Eye camera (YOLO exported to ONNX)
Done when: Phase 1 complete on the laptop
Frontend / Vision · Medium

Workstreams

Vision / ML		2 Debug view	3 Capture · label · fine-tune	3 Retrain	6 ONNX
Backend	1 Foundations	2 /ws/live	4 Scan mapping	5 Tracker + re-plan	
Frontend		2 Overlay	4 Scan UI	5 Guide UI	6 Arrows

Testing

LEVEL	WHAT	WHEN
Unit tests	Facelet mapping, matcher scoring, state machine transitions, solver wrapper	Every commit
Benchmarks	bench/ suite: solver, colours, tracking vs point error, YOLO accuracy	Every model or tracker change
Synthetic solve video	Rendered video of a full solve with random mistakes, streamed through /ws/live	Before merging Step 5
Real-camera checklist	5 scrambles × 3 lighting setups; record misses as new training data	End of Steps 3, 4, 5

RISKS AND EVIDENCE

Risks, open questions, sources

RISK	IMPACT	MITIGATION
YOLO is weaker on real video than on synthetic frames	Tracking gets "lost" often	Step 3 comes before scan/tracking work; capture failing cases and retrain; target < 5% error
Users rotate the whole cube mid-solve	Nothing matches → lost	Clear "home position" UI; home check; rescan is cheap (2 views)
Glare and fingers hide stickers	Wrong or unsure readings	Capped scoring, steady buffer, re-calibrated references
Server too slow on CPU-only laptops	< 10 fps	Vectorised matcher; skip frames; GPU/MPS when available
Licences: kociemba GPL-2.0, Ultralytics AGPL-3.0	Matters if sold or closed-source	Fine for internal use; decide before any release (alternatives exist)
sebastienlato/cube-solver has no licence	Cannot copy its code	We use the method only (own implementation, already prototyped)

Open questions for the team

1. Licensing: is GPL / AGPL acceptable for how this will be shipped?
2. Hardware: which laptops must reach 10 fps? Is a GPU/Apple-silicon minimum acceptable?
3. Who records and labels the 300–500 real frames for Step 3, and with which cubes (sticker vs stickerless)?
4. Do we keep the 6-face scan as a fallback, or ship the 2-view scan only?

Benchmark evidence

BENCHMARK	RESULT	SETUP
Solvers	kociemba 20.7 moves, 100% solved; current app 110 moves	200 random cubes, replayed on our cube model
Sticker finding, cube held freely	best classical 38.5% of frames	200 synthetic frames per condition
Sticker finding, corner view	classical 0–2.7%; YOLO 100%	150 / 300 synthetic frames
Colours (combined)	98% / 88.7% cubes fully right	150 cubes per lighting level
Tracking with YOLO	98.7% / 90.3% correct, 0% wrong	300 before/after trials per lighting level
Real photos	rubiks-cube-tracker and qbr (multi-scale) read 6/6 faces	dwalton76 test photos, one real cube

Full numbers and charts: **Benchmark_Report.pdf**. Caveat: almost all test images are synthetic renders (realistic lighting, glare, fingers, blur, noise); real-world results will be lower until Step 3 is done.

Open-source components

COMPONENT	USED FOR	LICENCE
kociemba (muodov/kociemba)	Solver	GPL-2.0
rubiks-color-resolver (dwalton76)	Colour fallback	MIT
Ultralytics YOLO	Cube keypoint detector	AGPL-3.0
sebastienlato/cube-solver	7-point / homography method (idea only)	None stated
OpenCV, FastAPI, NumPy	Imaging, server, maths	Apache-2.0 / MIT / BSD